

Chapter 9. Synthetic Control

In the previous chapter you learned about the advantages of panel data for causal identification. Namely, the fact that you could not only compare units with each other, but also with their former selves, allows you to estimate counterfactuals Y_0 with more plausible assumptions. You also learned about difference-in-differences (DID)—and many variations of it—one of the many causal inference tools that leverage panel data. By relying on similar (parallel) growth trajectories between treated and control, DID was able to identify the treatment effect even if the levels of Y_0 between treated and control were different. In this chapter, you'll learn another popular technique for panel datasets: synthetic control (SC).

While DID works great if you have a relatively large number of units N compared to time periods T , it falls short when the reverse is true. In contrast, synthetic control was designed to work with very few, even one, treatment unit. The idea behind it is quite simple: combine the control units in order to craft a synthetic control that would approximate the behavior of the treated units in the absence of treatment. By doing that, it avoids making a parallel trend assumption as the synthetic control, when well crafted, won't be just parallel, but perfectly overlapping with the counterfactual $E[Y_0|D = 1]$.

At the end of this chapter, you'll also learn how to combine both DID and SC. This combined estimator is not only very powerful, but, most importantly, it will give you a whole new perspective on difference-in-differences and synthetic control in particular, and panel data methods in general.

Online Marketing Dataset

As a use case for synthetic control, you'll be working with an online marketing dataset. Online marketing allows for better tracking than offline marketing, but it doesn't mean it comes without its challenges for causal inference. For instance, it is true that online marketing allows better attribution: you know if a customer reached your product through some paid marketing link. But that doesn't mean you know what would have happened to that customer if they didn't see your online ad. Perhaps customers only came because they saw the ad, in which case it is bringing in extra customers. But perhaps customers would have come either way and the fact that they did through the paid link was just because that link was at the top of the page.

Since attribution is not the same as incrementality, and since you can't randomize who gets to see your ads, treating entire geographies and doing some sort of panel data analysis, much like in the previous chapter, is also a good idea for online marketing. Hence, the data you have here is not much

different from the data you saw in the previous chapter. Once again, you have the city as the unit and date as the time dimension, a treated column, which marks if the city is eventually treated, and a post-treatment column, which marks the post-intervention period. You also have some auxiliary columns, like the population in that city (recorded in 2013, so fixed in time) and the state:

```
In [1]: import pandas as pd
        import numpy as np

        df = (pd.read_csv("../data/online_mkt.csv",
                           dtype={"date": "datetime64[ns]"}).
               .astype({"date": "datetime64[ns]"}))

        df.head()
```

	app_download	population	city
0	3066.0	12396372	sao_pau
1	2701.0	12396372	sao_pau
2	1927.0	12396372	sao_pau
3	1451.0	12396372	sao_pau
4	1248.0	12396372	sao_pau

Here, the outcome variable is daily app downloads and the treatment is having the marketing campaign turned on for that city. The treatment is implemented on the treated cities at the same time, which means you have a simple block design. The catch here is that you have a much smaller number of treated units—only three cities:

```
In [2]: treated = list(df.query("treated==1")["city",  
      treated
```

```
Out[2]: ['sao_paulo', 'porto_alegre', 'joao_pes:
```

If you paid attention to the population column in the data frame, you might have noticed that one of those cities, Sao Paulo, has a massive population of over 12MM. In fact, it is one of the biggest cities in the world! This also means that the number of app downloads in Sao Paulo will be a lot larger than in other cities, which poses some challenges. It is very hard to combine other cities to make a synthetic control that matches Sao Paulo's downloads. This issue is exacerbated here, but, in general, entire markets will have different sizes, making comparison across them difficult. Hence, a common approach is to *normalize the outcome by the market size*. This means dividing the number of app downloads by the city's population to create a normalized version of the outcome. This new outcome, `app_download_pct`, represents the number of daily downloads as a percentage of the market size:

```
In [3]: df_norm = df.assign(  
        app_download_pct = 100*df["app_downl  
        )  
  
        df_norm.head()
```

	app_download	population	city
0	3066.0	12396372	sao_pau
1	2701.0	12396372	sao_pau
2	1927.0	12396372	sao_pau
3	1451.0	12396372	sao_pau
4	1248.0	12396372	sao_pau

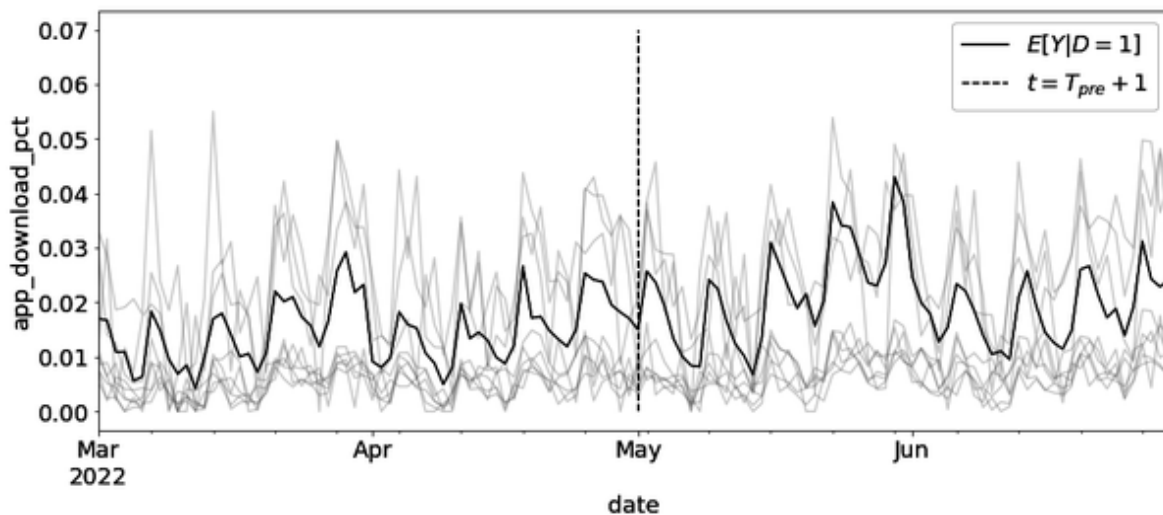
Moving forward with your exploratory analysis, you'll find out that an online marketing campaign was launched for those cities in 2022-05-01. The campaign also stayed on for the remainder of the analyzed time window:

```
In [4]: tr_period = df_norm.query("post==1")["date"]
        tr_period
```

```
Out[4]: Timestamp('2022-05-01 00:00:00')
```

This is a good time to review some of the panel notation you'll be using. Recall that, to avoid confusion, I'll use D to denote the treatment variable and t to denote time. T will be the number of periods, with T_{pre} being the number of periods before the intervention and T_{post} , the number of periods after the intervention. Hence, the treatment takes place when $D = 1$ and $t > T_{pre}$. To declutter, I'll sometimes use a post dummy to indicate $t > T_{pre}$. The combination of both treated and post-treatment will be represented by $W_{it} = D_i * Post_t$.

For you to get a sense of what this data looks like, the following plot shows the evolution of the average outcome of the three treated cities, and a sample of control cities, in the background, in light gray. The beginning of the post-treatment period is marked by a horizontal dashed line:



Looking at this plot, you can kind of see an increase in the treated units' outcome after the intervention, but it is not

100% clear. To be more precise, you would have to estimate the counterfactual and compare it to the observed outcome to get an estimate for average treatment effect on the treated (ATT):

$$ATT = E[Y|D = 1, Post = 1] - E[Y_0|D = 1, Post = 1]$$

Here is where synthetic control comes in. It is an incredibly *clever way to use (but not condition on) past outcomes in order to estimate $E[Y_0|D = 1, Post = 1]$* .

Matrix Representation

In the previous chapter, I showed you an image that represents panel data as a matrix, where one dimension is the time period and the other dimension denotes the units.

Synthetic control makes explicit use of that matrix, so it is worth reviewing it. Let's say that the rows of the matrix are the time periods and the columns of the matrix are the cities (units). You can represent the treatment assignment with four blocks:

$$W = \begin{bmatrix} 0_{pre,co} & 0_{pre,tr} \\ 0_{post,co} & 1_{post,tr} \end{bmatrix}$$

The first block in your matrix (top left) corresponds to the control units prior to the treatment period; the second one (top right) corresponds to the treated units prior to the treatment period; the third block (bottom left) contains the

control units after the treatment period; and the fourth block (bottom right) is the treated unit after the treatment period. The treatment indicator w_{ti} is zero everywhere except for the block with the treated units after the treatment period (bottom right).

This assignment matrix will lead to the following observed potential outcome matrix:

$$Y = \begin{bmatrix} Y(0)_{pre,co} & Y(0)_{pre,tr} \\ Y(0)_{post,co} & Y(1)_{post,tr} \end{bmatrix}$$

Again, notice how the post-treatment period is on the bottom and the treated units are to the right. Your goal is to estimate the $ATT = Y(1)_{post,tr} - Y(0)_{post,tr}$. For that, you need to somehow estimate the missing potential outcome $Y(0)_{post,tr}$, which is not observed. In words, you need to know what would have happened to the treated units at the post-treatment period had they not been treated. To achieve that, you would ideally leverage all the other three blocks at your disposal, $Y(0)_{pre,co}$, $Y(0)_{pre,tr}$, and $Y(0)_{post,co}$. Before I show you how synthetic control does that, let's create a function to represent the data in this matrix format.

The following code uses the `.pivot()` method to reshape the data frame so that you end up with one row per time period (date) and one column per city, while the outcome becomes the values of the matrix. Then, it partitions the

matrix into treated and control units. It further partitions them into a pre- and post-intervention period:

```
In [5]: def reshape_sc_data(df: pd.DataFrame,
                             geo_col: str,
                             time_col: str,
                             y_col: str,
                             tr_geos: str,
                             tr_start: str):

    df_pivot = df.pivot(time_col, geo_col, y_col)

    y_co = df_pivot.drop(columns=tr_geos)
    y_tr = df_pivot[tr_geos]

    y_pre_co = y_co[df_pivot.index < tr_start]
    y_pre_tr = y_tr[df_pivot.index < tr_start]

    y_post_co = y_co[df_pivot.index >= tr_start]
    y_post_tr = y_tr[df_pivot.index >= tr_start]

    return y_pre_co, y_pre_tr, y_post_co, y_post_tr
```

You'll use this four-block matrix representation throughout the chapter. If you ever forget what you are working with, just come back to this function. To see how it works, passing

`df_norm` to `reshape_sc_data` returns you the

Ys in matrix format. Here are the first five rows of

```
y_pre_tr :
```

```
In [6]: y_pre_co, y_pre_tr, y_post_co, y_post_tr =\
        df_norm,\
        geo_col="city",\
        time_col="date",\
        y_col="app_download_pct",\
        tr_geos=treated,\
        tr_start=str(tr_period)\
        )

y_pre_tr.head()
```

city	sao_paulo	porto_alegre	joao_pes
date			
2022-03-01	0.024733	0.004288	0.022039
2022-03-02	0.021789	0.008107	0.020344
2022-03-03	0.015545	0.004891	0.012352
2022-03-04	0.011705	0.002948	0.018285
2022-03-05	0.010067	0.006767	0.000000

Synthetic Control as Horizontal Regression

The main idea behind synthetic control is quite simple. Using the pre-treatment period, you'll find a way to combine the control units to approximate the average outcome of the treated units. In mathematical terms, this can be framed as an optimization problem, where you'll look for unit weights ω_i (not to be confused with $w_{it} = Post_t * D_i$) such that, when

you multiply each weight by its unit's outcome, $\omega_i y_i$, you get something resembling the treated unit's outcome:

$$\hat{\omega}^{sc} = \underset{\omega}{\operatorname{argmin}} \quad ||\bar{y}_{pre,tr} - Y_{pre,co}\omega_{co}||^2$$

Then, to estimate $E[Y(0)|D = 1, Post = 1]$ and get the ATT estimate, you can use the synthetic control $Y_{post,co}\omega_{co}$.

If this seems a bit cryptic, perhaps a good alternative explanation is one that compares synthetic control to a more familiar tool: linear regression. Recall that regression could also be represented by an optimization problem where the goal was to minimize the (squared) difference between the outcome and a linear combination of the covariates X :

$$\beta^* = \underset{\beta}{\operatorname{argmin}} \quad ||Y_i - X_i'\beta||^2$$

OUTCOME MODELING

Here, you can draw a parallel between synthetic control and potential outcome modeling, which you saw in [Chapter 5](#), when reading about Doubly Robust Estimation. There, you also had to build a regression model that was estimated in the control group. Then, you used that model to impute the missing potential outcome, Y_0 , for those that were treated. The idea is pretty much the same here.

As you can see, both objectives are identical! This means that synthetic control is nothing more than *a regression that uses the outcome of the control as features to try to predict the average outcome of the treated units*. The trick is that it does this by using only the pre-intervention period so that the regression estimates $E[Y_0|D = 1]$.

In fact, to prove my point, let's use OLS to build a synthetic control right now. All you have to do is to use

`y_pre_co` as if it was the covariate matrix X and the column average of `y_pre_tr` as the outcome y . Once you fit this model, the weights can be extracted with

`.coef_`:

```
In [7]: from sklearn.linear_model import LinearRegression

model = LinearRegression(fit_intercept=True)
model.fit(y_pre_co, y_pre_tr.mean(axis=0))

# extract the weights
weights_lr = model.coef_
weights_lr.round(3)
```

```
Out[7]: array([-0.65 , -0.058, -0.239,  0.971,
               0.102,  0.106,  0.074,  0.079,
              -0.014,  0.132,  0.115,  0.094,
              -0.476, -0.11 ,  0.158, -0.002,
              -0.047,  0.089, -0.057,  0.429,
              -0.128,  0.128, -0.205,  0.088,
```

As you can see, you have one weight for each control city. Usually, regression is used when you have a bunch of units

(large N), which allows you to use the units as the rows and the covariates as the columns. But synthetic control is designed to work when you have relatively few units, but a larger time horizon T_{pre} . In order to do that, SC quite literally flips the data on its head, using the units as if they were covariates. This is why synthetic control is also called horizontal regression (see [Figure 9-1](#)).

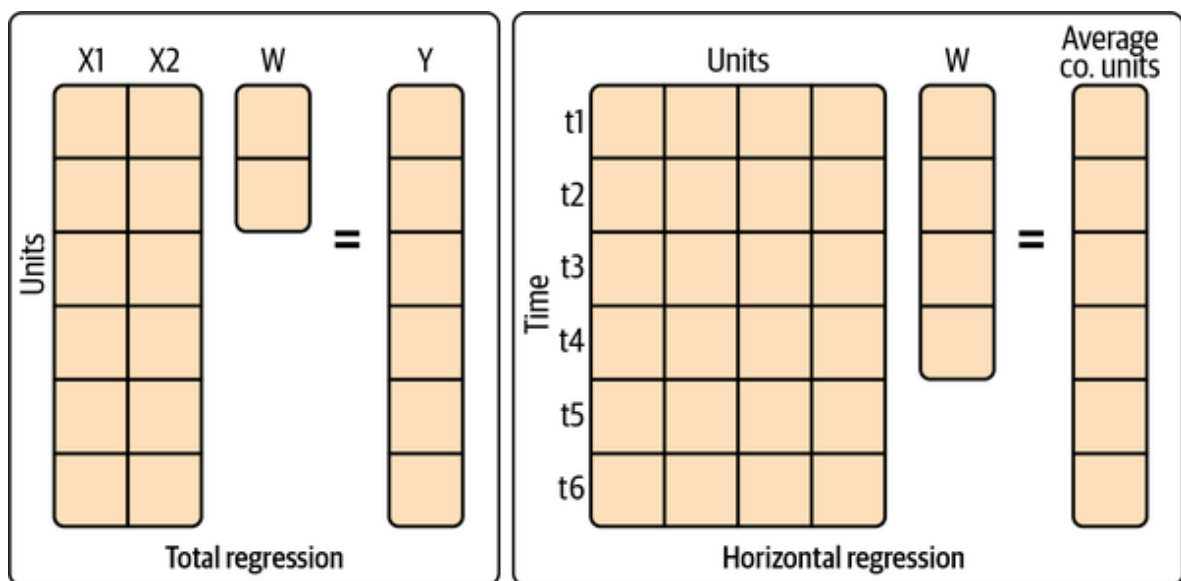


Figure 9-1. In traditional regression, units are the rows of the regression table; in horizontal regression, the rows are time periods and the units are the columns

Once you've estimated your regression parameters (or weights), you can use them to predict what $E[Y_0|D = 1]$ would look like, not only on the pre-intervention period, but on the entire time horizon:

```
In [8]: # same as y0_tr_hat = model.predict(y_pre)
        y0_tr_hat = y_post_co.dot(weights_lr)
```

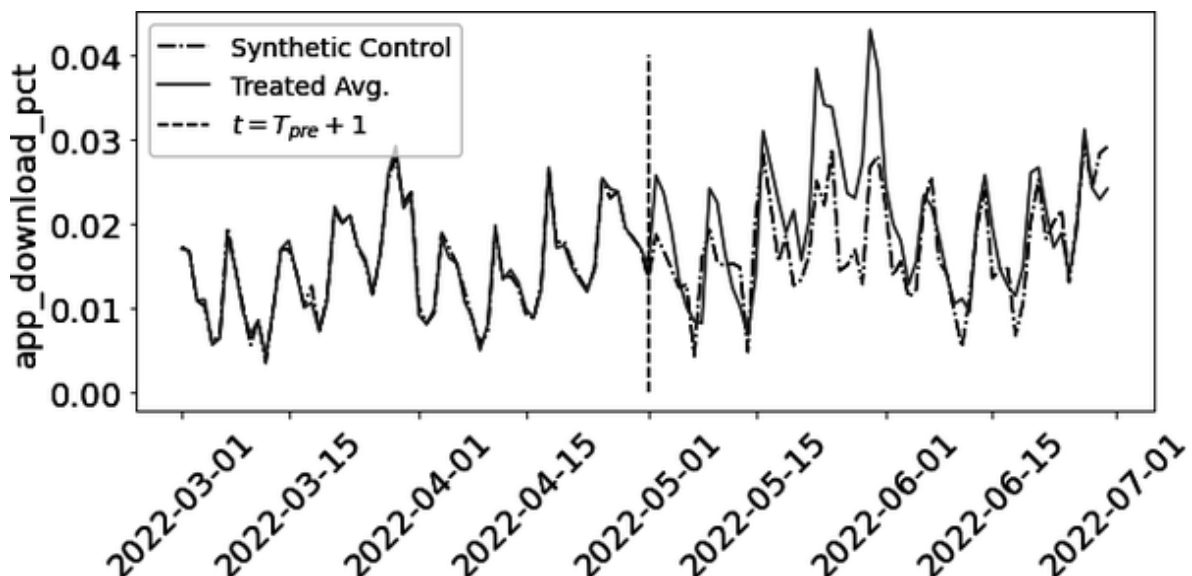
Here, `y0_tr_hat` can be seen as a synthetic control: a combination of control units that come together to approximate the behavior of the treated units' average, had they not been treated.

AVERAGE OF SYNTHETIC CONTROLS

Alternatively, instead of finding one synthetic control to replicate the average outcome of the treated units, you could also fit one synthetic control for each treatment unit individually and then average the synthetic controls:

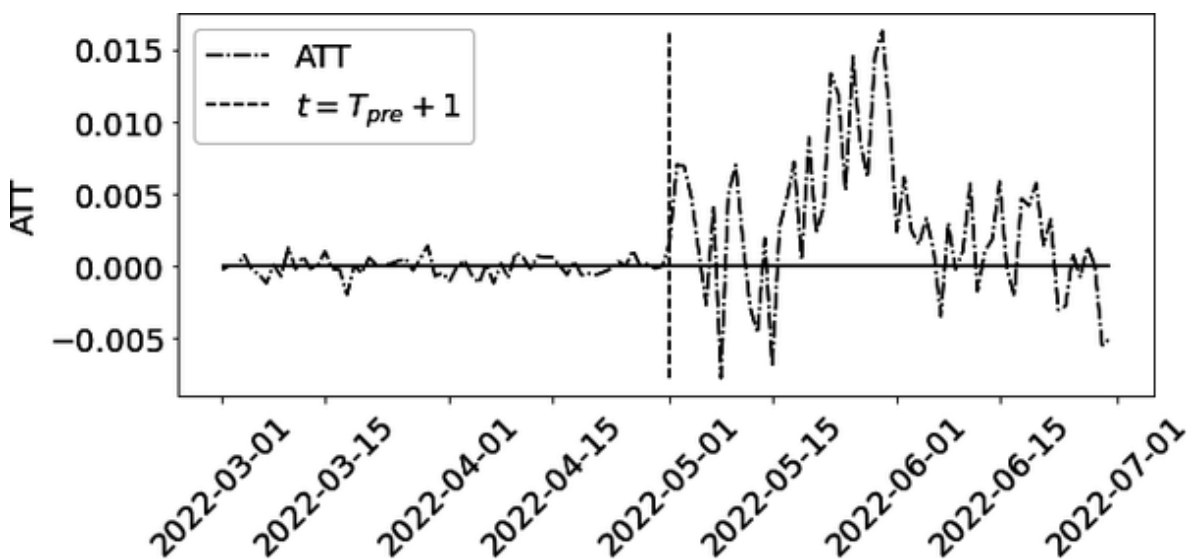
```
model = LinearRegression(fit_intercept=False)
model.fit(y_pre_co, y_pre_tr)
y0_tr_hat = model.predict(y_co).mean(axis=1)
```

If you plot this synthetic control alongside the observed outcome, you'll get this:



Notice how the predicted value (the synthetic control) is below the actual outcome of the treated units. It means that the observed outcome was higher than you've estimated it to be, had the treatment not taken place. This indicates a positive marketing effect from the online marketing campaign. You can compute that ATT estimate by contrasting the observed outcome with the synthetic control:

```
In [9]: att = y_post_tr.mean(axis=1) - y0_tr_hat
```



The plot presents a couple of intriguing aspects. Firstly, it suggests that the effect takes some time to reach its peak before gradually declining. The gradual increase is frequently observed in marketing since it usually requires time for individuals to take action after seeing an advertisement.

Additionally, the effect wearing off can often be attributed to a novelty effect that gradually fades over time.

The second interesting thing is the size of the ATT in the pre-intervention period. In that time frame, the ATT can be interpreted simply as the residual (in sample error) from your OLS model. You might think that it being close to zero is a good thing; after all, you don't want to see an effect prior to the treatment (anticipation). But there is more to it. The fact that the pre-intervention error is incredibly low can also mean that the OLS model is probably overfitting. As a result, the out-of-sample prediction, which should estimate $E[Y_0|D = 1, Post = 1]$, might be off.

This is why simple regression is not commonly used as a method to build synthetic controls. Because of the relatively large number of columns (control cities), it tends to overfit, not generalizing to the post-intervention period. For this reason, the original synthetic control method is not a simple regression, but one that imposes some reasonable and intuitive constraints.

Canonical Synthetic Control

The canonical synthetic control formulation imposes two constraints on the regression model:

1. That the weights are all positive

2. That the weights add up to one

Mathematically, the optimization objective becomes:

$$\begin{aligned}\hat{\omega}^{sc} = \operatorname{argmin}_{\omega} \quad & \|\bar{\mathbf{y}}_{pre,tr} - \mathbf{Y}_{pre,co}\omega_{co}\|^2 \\ \text{s.t} \quad & \sum \omega_i = 1 \text{ and } \omega_i > 0 \quad \forall i\end{aligned}$$

The idea behind the constraints is to force the synthetic control to be a convex combination of the treated units, *avoiding extrapolation*. This means that if the treated unit has an outcome greater (or lower) than all the control units, this canonical formulation won't be able to craft a synthetic control to recover $E[Y_0|D = 1]$. You can view this as a limitation, but it is actually meant as a guardrail. It's a way of saying that the treatment units you are trying to reconstruct are very different from the ones in the control group and therefore you shouldn't even try.

To code the canonical version of SC, you can use convex optimization software, like `cvxpy`. `cvxpy` allows you to define an optimization objective with

`cp.Minimize`. For SC, you want to minimize the squared error, so `cp.Minimize(cp.sum_squares(y_co_pre@w - y_tr_pre))`. It also allows you to pass optimization constraints. Here, you want all the `w`s to be nonnegative and `np.sum(w)==1`.

In the following code, I'm building the synthetic control model following scikit-learn's boilerplate. To do that, you can extend

`BaseEstimator` and `RegressorMixin` and define a `.fit` and a `.predict` method. The rest of the code, like `check_X_y`, `check_array`, and `check_is_fitted` are just some standard checks you don't need to worry about:

```
In [10]: from sklearn.base import BaseEstimator,
          from sklearn.utils.validation import (
          import cvxpy as cp

          class SyntheticControl(BaseEstimator,

          def __init__(self,):
              pass

          def fit(self, y_pre_co, y_pre_tr):

              y_pre_co, y_pre_tr = check_X_y

              w = cp.Variable(y_pre_co.shape

              objective = cp.Minimize(cp.sum_
              constraints = [cp.sum(w) == 1,

              problem = cp.Problem(objective,

              self.loss_ = problem.solve(verb
              self.w_ = w.value
```

```

        self.is_fitted_ = True
        return self

    def predict(self, y_co):

        check_is_fitted(self)
        y_co = check_array(y_co)

        return y_co @ self.w_

```

Having defined the `SyntheticControl` class, you can use it pretty much like you used `LinearRegression` before. Notice that I'm storing the final loss of the estimated model. This will come in handy if you want to incorporate covariates in your model, as you'll soon see. Also, after the model is fitted, you can access the weights with `.w_`:

```

In [11]: model = SyntheticControl()
         model.fit(y_pre_co, y_pre_tr.mean(axis=0))

         # extract the weights
         model.w_.round(3)

```

```

Out[11]: array([-0.    , -0.    , -0.    , -0.    ,
                0.083,  0.01  , -0.    , -0.    ,

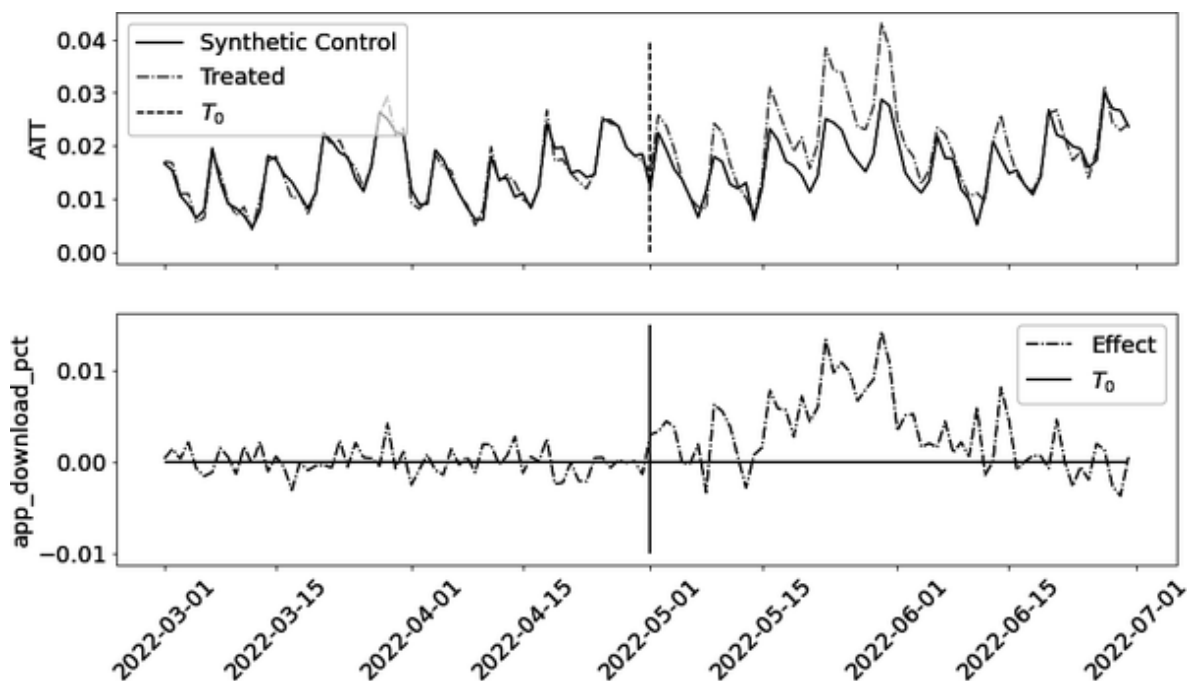
```

```

0.061, 0.123, 0.008, 0.074,
-0.    , -0.    , -0.    , -0.    ,
-0.    , -0.    , 0.    , 0.046,
0.    , -0.    , -0.    , 0.088,

```

Notice another interesting thing: the convexity constraints you've imposed give a sparse solution to the optimization problem. Only a handful of cities are used to craft the final synthetic control. From this point onward, it's exactly the same thing as before. You can make predictions on the entire dataset to obtain the synthetic control estimate for $E[Y_0|D = 1]$ and use that to get the ATT estimate:



Compare this ATT plot with the one you got earlier, using unconstrained regression. Now, the training (pre-treatment)

error is a bit larger, but the ATT is less noisy. This is regularization in action.

REGULARIZED REGRESSION

Once you realize that synthetic control is just horizontal regression, you can find other ways to regularize it. For instance, you can use lasso or ridge regression. Still, it makes a lot of sense to not allow negative weights, especially if the units are geographies, as the outcome tends to be positively correlated among those.

SYNTHETIC CONTROL ASSUMPTIONS

Just like with difference-in-differences, you also have to assume no anticipation of the treatment and no spillovers when using synthetic control. The main difference between the two methods is the parametric assumption about the potential outcome model. In diff-in-diff, you had to assume that the trend in Y_0 for the treated units was parallel to the Y_0 trend from the control units. Synthetic control, on the other hand, allows a more flexible (but also more complicated) model for that potential outcome: a vector autoregressive model or a linear factor model. For the factor model:

$$Y_{0it} = \lambda_i \hat{a} \mathbf{f}_t + e_{it},$$

if you set $\lambda_1 = 1$, $f_1 = \beta_t$ and $\lambda_i = \alpha_i$, $f_1 = 1$, you can see that it becomes a generalization of the two-way-fixed-effects model $Y_{0it} = \alpha_i + \beta_t + e_{it}$.

In the paper “Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects,” Alberto Abadie argues that if the potential outcome follows either the vector autoregressive or the linear factor model *and* the synthetic control matches the treated unit, then the synthetic control method produces an unbiased estimate to the ATT. In practice, that SC can only approximate the treated unit, so some bias is to be expected.

Synthetic Control with Covariants

Usually, synthetic control uses just the pre-treatment outcome from the control units as features to predict $\bar{Y}_{tr}$. That's because those tend to be the most predictive feature at your disposal. However, you might wish to include some additional covariates in the model, if you think they have a good predictive power. This is quite rare, though, so, if you are short of time, you can skip this section.

Let's say you somehow manage to get data on the daily number of downloads from your main competitor, which you've also normalized by the market size,

`comp_download_pct`. You think this covariate is a good predictor of $\bar{Y}_{tr}$, so you want to include it in the synthetic control model:

```
In [12]: df_norm_cov = (pd.read_csv("./data/only_norm.csv")
          .astype({"date": "datetime64[ns]"}))

          df_norm_cov.head()
```

	app_download	city	date
0	3066.0	sao_paulo	2022-03
1	2701.0	sao_paulo	2022-03
2	1927.0	sao_paulo	2022-03
3	1451.0	sao_paulo	2022-03
4	1248.0	sao_paulo	2022-03

In mathematical notation, you want to construct a synthetic control such that the weight w_i is not only multiplied by y_{co} , but also by this extra covariate, x_{co} , in order to approximate $\bar{Y}_{tr}$. The issue here is that x_{co} and y_{co} might be in completely different scales or one can be more predictive than the other, which is why you need to multiply each covariate, including y_{co} , by a scaling factor v , *before* solving the SC optimization problem. To take that into account, you can rewrite the objective in terms of covariates X , treating y_{co} as just another covariate:

$$\hat{w}^{sc} = \underset{\omega}{\operatorname{argmin}} \left\| \bar{\mathbf{y}}_{pre,tr} - \sum v_k^* \mathbf{X}_{k,pre,co} \omega_{co} \right\|^2$$

$$\text{s.t } \sum \omega_i = 1 \text{ and } \omega_i > 0 \ \forall \ i$$

However, this objective doesn't tell you how to find the optimal v . In order to do that, you'll have to wrap the entire synthetic control into yet another optimization objective. If it sounds complicated, don't worry. It is a lot easier to understand it in code. First, let's create the X matrix for both covariates, `comp_download_pct` and $Y_{pre,co}$,

```
app_download_pct :
```

```
In [13]: from toolz import partial

         reshaper = partial(reshape_sc_data,
                             df=df_norm_cov,
                             geo_col="city",
                             time_col="date",
                             tr_geos=treated,
                             tr_start=str(tr_peri

         y_pre_co, y_pre_tr, y_post_co, y_post_tr =
             y_col="app_download_pct"
         )

         x_pre_co, _, x_post_co, _ = reshaper(y_
```

Next, let's write a function which, when given a list of vs , one for each covariate, returns the synthetic control weights and the optimization loss. Remember that you can access the

objective loss from a fitted `SyntheticControl` model with `.loss_`.

To check if this is working, you can pass `[1, 0]` as the *vs* and `[y_pre_co, x_pre_co]` as the covariate list. You should get back the original synthetic control, since the extra covariate has zero weight in this case:

```
In [14]: def find_w_given_vs(vs, x_co_list, y_tr_pre):
          X_times_v = sum([x*v for x, v in zip(x_co_list, vs)])

          model = SyntheticControl()
          model.fit(X_times_v, y_tr_pre)

          return {"loss": model.loss_, "w": model.w_}

          find_w_given_vs([1, 0],
                          [y_pre_co, x_pre_co],
                          y_pre_tr.mean(axis=1))
```

```
Out[14]: array([-0.    , -0.    ,  0.    , -0.    ,
                0.085,  0.003, -0.    , -0.    ,
                0.062,  0.121, -0.    ,  0.072,
               -0.    , -0.    ,  0.    , -0.    ,
                0.    , -0.    ,  0.    ,  0.022,
               -0.    , -0.    , -0.    ,  0.088,
```

Finally, you can wrap `find_w_given_vs` in a function that just takes the array of vs and returns the optimization loss. Then, you can then pass this function to `scipy minimize` function, which will iteratively look for the best vs and return them to you:

```
In [15]: from scipy.optimize import minimize

def v_loss(vs):
    return find_w_given_vs(vs,
                           [y_pre_co, y_pre_tr.meas],
                           y_pre_tr.meas)

v_solution = minimize(v_loss, [0, 0], method='Nelder-Mead')
v_solution.x
```

```
Out[15]: array([1.88034589, 0.00269853])
```

With the optimal vs , you can go back to

`find_w_given_vs` to obtain the synthetic control weights that take the covariates into account. One thing to notice, though, is that the final solution is not much different from the one without covariants. This is not surprising, since

the optimal v for the `comp_download_pct` covariate is a very small number and it is not on a much larger scale than

`app_download_pct` :

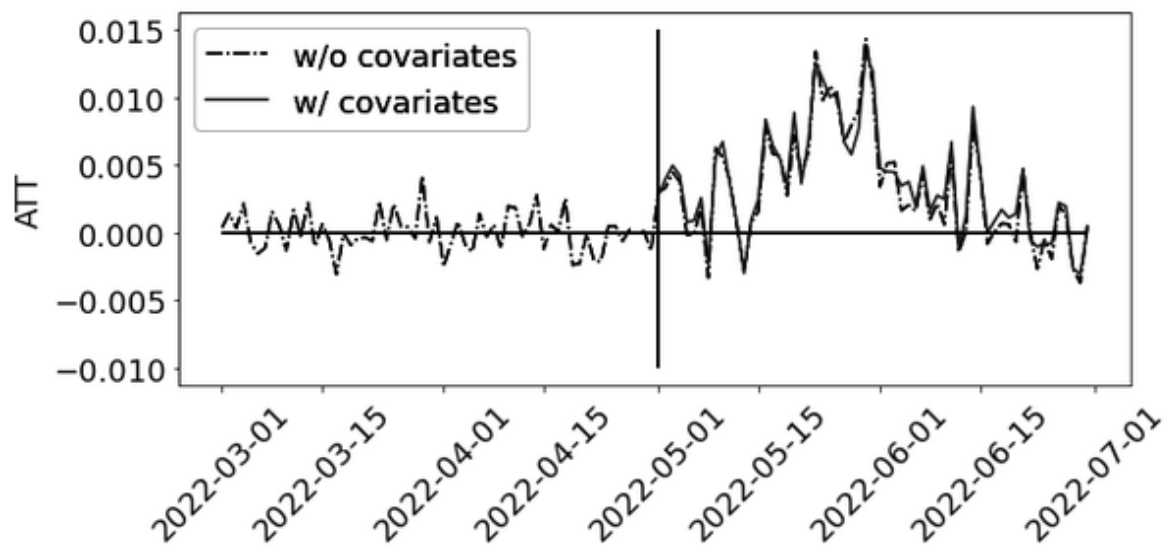
```
In [16]: w_cov = find_w_given_vs(v_solution.x,
                                   [y_pre_co, x_pre_co,
                                   y_pre_tr.mean(axis=1),
                                   x_pre_tr.mean(axis=1)],
                                   w_cov)
```

```
Out[16]: array([-0.      , -0.      ,  0.      , -0.      ,
                 0.033,   0.      , -0.      ,  0.034,
                 0.016,   0.047,   0.03   ,  0.01   ,
                 -0.      ,  0.      , -0.      ,  0.      ,
                 0.078,   0.007,   0.      ,  0.      ,
                 -0.      ,  0.201,   0.      ,  0.035,
```

With these weights, you can make predictions for $Y(0)$ and obtain the final ATT estimate that considers covariates:

```
In [17]: y0_hat = sum([x*v for x, v
                        in zip([y_post_co, x_post_co,
                                y_post_tr.mean(axis=1),
                                x_post_tr.mean(axis=1)],
                               w_cov)])
att = y_post_tr.mean(axis=1) - y0_hat
```

The following plot shows the resulting ATT, alongside the ATT estimate from the canonical SC, without covariates. As you can see, both are pretty similar:



Although not hard, including covariates requires a fair amount of extra complexity. For this reason and due to the fact that $Y_{pre,co}$ tends to be enough to predict Y_{tr} , I usually don't bother with adding covariates. But maybe you can find very predictive features that would justify it.

GENERIC HORIZONTAL REGRESSION

A simpler way to add covariates is to just concatenate any additional time series you deem worthy as a column to $\mathbf{Y}_{pre,co}$. This would be equivalent to adding additional covariates in the horizontal regression:

$$[\mathbf{Y}_{pre,co} | \mathbf{X}_{pre,co}] \omega$$

This wouldn't be a synthetic control in the strict sense of the words, since you are now estimating $E[Y(0)_{tr}]$ with the control units *and* those extra time series. As a result, you'll end up with weights not only for the units, but for each additional column that you've concatenated.

Debiasing Synthetic Control

Much like with powerful machine learning models, these prediction techniques are prone to overfitting, especially when the number of pre-treatment periods T_{pre} is small. Even the constraints imposed on the canonical synthetic control doesn't solve that completely. As a result, *SC is known to be biased*. To understand that, let's redefine the ATT as the average across time in the post-intervention periods:

$$ATT = \frac{1}{T_1} \sum_{t=T_0+1}^T (\bar{Y}_{1t} - \bar{Y}_{0t}),$$

where T_0 and T_1 are the size of the pre- and post-intervention periods, respectively, and $\bar{Y}_{dt}$ is the average potential outcome of the treated units. This simply averages the ATT for each individual post-intervention period into a single number, making it easier to work with it. Now, to check for bias in the SC method, you can compare that single number against its estimate. [Figure 9-2](#) shows that bias.

I'm simulating a bunch of data following the synthetic control specification—with the treated unit being a weighted combination of some control units. Here, $N = 16$ and $T_{pre} = 15$, so there are more columns than rows in the horizontal regression. Also, the true ATT is zero. Still, the resulting distribution of ATT estimates you get with synthetic control is not centered at zero, showing how it is indeed biased.

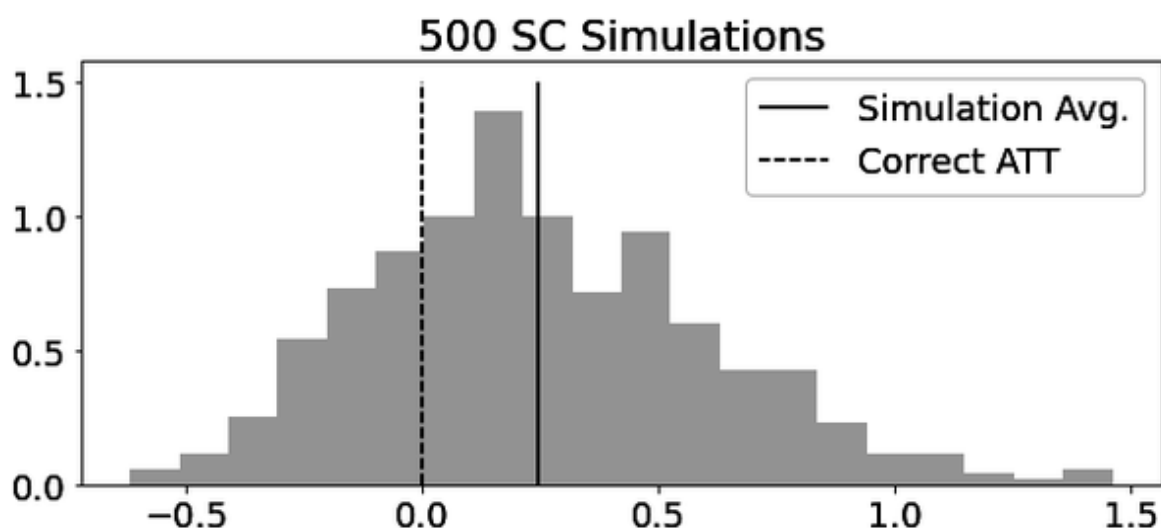


Figure 9-2. Synthetic control estimator is not unbiased, as it rarely manages to correctly specify the outcome model

Fortunately, you learned how to solve for overfitting bias in [Chapter 7](#), when you saw Double/Debiased ML. The answer lies in *cross-fitting*. The idea is to partition the pre-intervention period into K blocks, each of size $\min\{T_{pre}/K, T_{post}\}$ (the reason for this `min` function will become clear soon. For now, just trust me). Then, you'll treat each block as a hold-out set and fit a synthetic control model on $Y_{pre,co}^{-k}$ and $Y_{pre,tr}^{-k}$, where $-k$ means you'll drop the block from training. This step will give you weights $\hat{\omega}^{-k}$. Next, you'll use those weights to make out-of-sample predictions using the held-out data $Y_{pre,co}^k$. The average difference between the predictions and the observed values in the hold-out data is an estimate for the bias:

$$\widehat{Bias}^k = avg\left(\mathbf{Y}_{pre,tr}^k - \mathbf{Y}_{pre,co}^k \hat{\omega}^{-k}\right),$$

which means you can use it to adjust the ATT estimate:

$$\widehat{ATT}^k = \mathbf{Y}_{post,tr} - \mathbf{Y}_{post,co} \hat{\omega}^{-k} - \widehat{Bias}^k$$

Notice that this will give you K different *ATT*s. You can average them out to get a final ATT estimate.

Now, let's put this into Python code. The trickiest part here is to define the blocks. So, to make it more clear, let's go over a simple example, shown in [Figure 9-3](#).

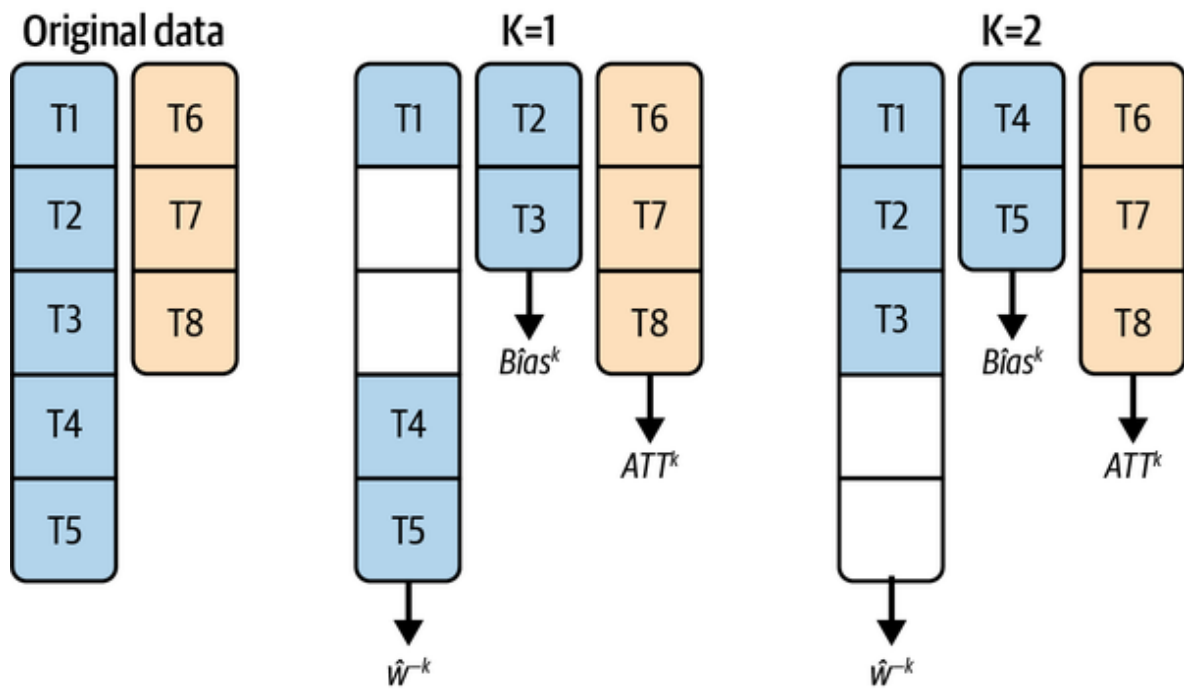


Figure 9-3. Cross-prediction can be used to estimate the bias and correct it

Suppose that you have five pre-intervention periods and three post-intervention periods and you want to build $K = 2$ blocks. The block size is 2.5, which is not an integer, so you have to floor it to two. This means you'll take two blocks of size two out of the pre-intervention period. 2×2 will give you four time periods, but you have five. So, I'm choosing to take the blocks from the end of the pre-intervention period, which will cause the first time period to never be removed. This is rather arbitrary, but doesn't have a huge impact on the whole procedure. You could also choose to trim the pre-intervention period to make it divisible by K .

Then, for each of the two blocks, you'll take it out of the training set, and estimate an SC model to obtain $\hat{W}^{-k}$. With these weights, you'll move to the removed block and estimate

the bias term. Finally, you'll use both weights and bias estimates to make an ATT estimate in the post-intervention period.

Even though it is a bit complicated to describe, it is fairly easy to get these blocks with NumPy. First, you'll index into the end of the pre-intervention period,

`y_pre_tr.index[-K*block_size:]` to get an index with exactly K blocks. Then, you can use `np.split` to break those indexes into K blocks. This will return an array with K rows, each one containing the index that you want to remove in each iteration. Once you have those blocks, you can iterate over them, fitting an SC model, estimating the bias and the ATT in the post-intervention period. The result can be stored in a data frame for convenient display:

```
In [18]: def debiased_sc_atts(y_pre_co, y_pre_tr,
                                y_post_co, y_post_tr,
                                block_size):
    block_size = int(min(np.floor(len(y_pre_co)/K),
                        block_size))
    blocks = np.split(y_pre_tr.index[-K*block_size:],
                      K,
                      np.arange(1, K+1)*block_size)

    def fold_effect(hold_out):
        model = SyntheticControl()
        model.fit(
            y_pre_co.drop(hold_out),
            y_pre_tr.drop(hold_out)
        )

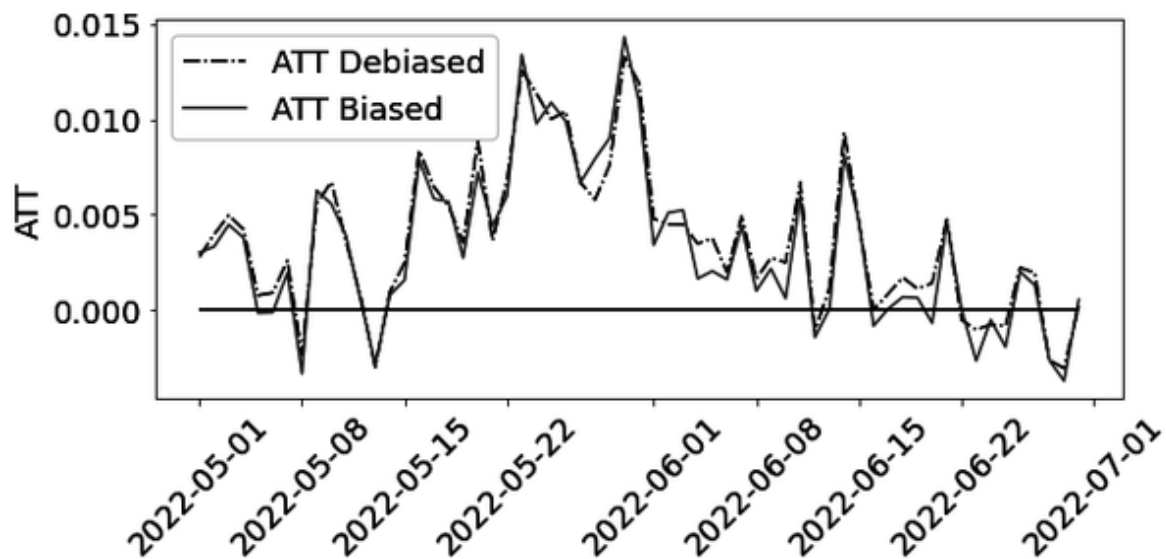
        bias_hat = np.mean(y_pre_tr.loc[hold_out] -
```


	0	1	2
date			
2022-05-01	0.003314	0.002475	0.003228
2022-05-02	0.003544	0.002844	0.003356
2022-05-03	0.004644	0.003698	0.004744
2022-05-04	0.004706	0.002866	0.003630
2022-05-05	0.000134	-0.000541	0.000243

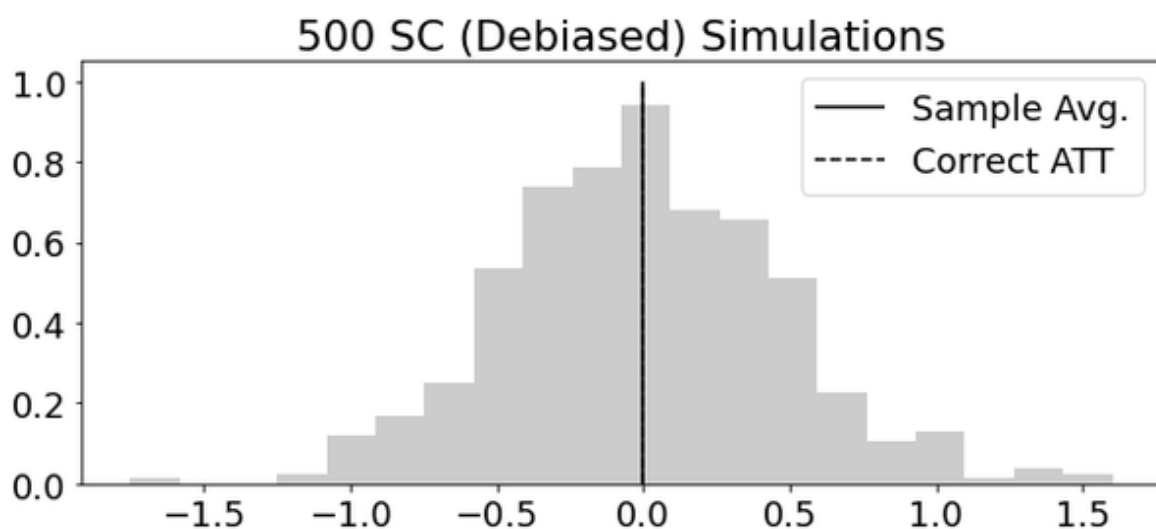
To get a final ATT estimate for each post-intervention period you can average out the columns,

`deb_atts.mean(axis=1)`, or, if you want a single ATT for the entire period, just average everything:

`deb_atts.mean(axis=1).mean()`. Plotting the debiased ATT alongside the canonical SC ATT you got earlier also shows that, for most parts, the debiasing increased the ATT estimate, although not by much:



It's difficult to see a difference in your marketing data, but to show why debiasing is important, I can redo the simulations from before, but now using the debiasing procedure. Now, the distribution of ATTs from the simulation has mean zero, as it should be:



Inference

The debiasing procedure is useful on its own, but there is also a second reason why it is interesting, which is to place a confidence interval around the synthetic control ATT estimate. Doing inference with synthetic control has proven to be a daunting task, mostly because there are usually very few control units, or even just one. The block bootstrap you learned in [Chapter 8](#) won't work here because a lot of the bootstrap samples will throw out all the treated units, making the ATT undefined.

Inference for the synthetic control method is an active area of research, which is rapidly changing. Over the last few years, many approaches have been proposed. Most of them rely on permuting the time dimension, as bootstrapping the units seems problematic. Here, I chose one of those methods that I find quite simple to implement and very computationally efficient. Especially if you already dealt with the debiasing part, because it uses it as a starting point. Just as a refresher, recall that debiasing gave you one ATT estimate for each of the K folds and for each post-intervention time period, which are represented as the columns of the following data frame:

```
In [20]: deb_atts.head()
```


	0	1	2
date			
2022-05-01	0.003314	0.002475	0.003228
2022-05-02	0.003544	0.002844	0.003356
2022-05-03	0.004644	0.003698	0.004744
2022-05-04	0.004706	0.002866	0.003630
2022-05-05	0.000134	-0.000541	0.000243

Now, let's say that you are interested in placing a confidence interval around the overall ATT estimate in the post-intervention period. To do so, the first thing you need is the $\widehat{ATT}$ itself. You can average out the rows of this data frame, which will give you a single ATT for each of the K folds. Then, you can take the average of that:

```
In [21]: atts_k = deb_atts.mean(axis=0).values
         att = np.mean(atts_k)

         print("atts_k:", atts_k)
```

```
print("ATT:", att)
```

```
Out[21]: atts_k: [0.00414872 0.00260513 0.003185]
ATT: 0.003313226501636449
```

Now, for the inference part. The idea here is to construct a standard error estimate based on each of the ATT^k :

$$\hat{\sigma} = \sqrt{1 + \frac{BlockSize * K}{T_{post}}} * \sqrt{\frac{1}{K-1} \sum_{k=1}^K (ATT^k - ATT)^2}$$

$$\widehat{SE} = \hat{\sigma} / \sqrt{K}$$

When coding this, you just need to be careful to use the sample standard deviation, which means passing

`ddof=1` to `np.std`:

```
In [22]: K = len(atts_k)
          T0 = len(y_pre_co)
          T1 = len(y_post_co)
          block_size = min(np.floor(T0/K), T1)

          se_hat=np.sqrt(1+((K*block_size)/T1))*np.std(atts_k, ddof=1)

          print("SE:", se_hat)
```

```
Out[22]: SE: 0.0006339596260850461
```

With that standard error, you can construct a test statistic $\widehat{ATT}/\widehat{SE}$ which, under the null hypothesis $H0 : ATT = 0$, has asymptotic t -distribution with $K - 1$ degrees of freedom. This means you can leverage it to construct a confidence interval using the t -distribution. For instance, here is how you can construct a 90% CI ($\alpha = 0.1$):

```
In [23]: from scipy.stats import t
         alpha = 0.1

         [att - t.ppf(1-alpha/2, K-1)*se_hat,
          att + t.ppf(1-alpha/2, K-1)*se_hat]
```

```
Out[23]: [0.0014620735349405393, 0.00516437946850461]
```

You might look at that K in the denominator of the standard error formula and be tempted to set it to a very large number. However, there is no free lunch here. Higher values of K result in smaller confidence intervals at the cost of lowering the coverage of those intervals. For high K s, the $1 - \alpha$ CIs will contain the true ATT less than $1 - \alpha$ of the time, especially

when the number of pre-treatment periods is small. In this case, a reasonable choice of K is 3. When T_0 is very large, compared to N , you can try larger values of K to decrease the length of the confidence interval.

Another important point is that this method does not apply to the treatment effect trajectory. That is, it can't be used for a per-period inference, as its theory requires both T_0 and T_1 to be relatively large.

SEE ALSO

This inference methods was proposed in the paper “A T-Test for Synthetic Controls,” by Victor Chernozhukov et al. If you want to perform per-period inference, the same authors have a complementary paper that proposes conformal inference for synthetic controls: “An Exact and Robust Conformal Inference Method for Counterfactual and Synthetic Controls.”

Synthetic Difference-in-Differences

To close this chapter, I wanted to give you yet another perspective on synthetic control, which is how it relates to difference-in-differences. By doing so, you'll also learn how to combine both methods into a single synthetic difference-in-differences (SDID) estimator. The idea here is quite simple. First, construct a synthetic control. Then, use it as the control unit in a DID setting. The end result is something much more interesting than the sum of its parts. First, the parallel assumption required for DID becomes much more plausible, since you are crafting a synthetic control for $E[Y(0)_{ti} | D = 1]$

. Second, because you are using DID, the synthetic control can focus on capturing just the trend of the treated unit, as it can have a different level of $Y(0)$. But first, before going into SDID, let's review some DID theory.

DID Refresher

In its canonical form, with one control block (never treated) and one treated block that gets treated all at the same time period, you could write DID with two-way fixed effects:

$$\hat{\tau}^{did} = \underset{\mu, \alpha, \beta, \tau}{argmin} \left\{ \sum_{n=1}^N \sum_{t=1}^T (Y_{it} - \mu + \alpha_i + \beta_t + \tau W_{it})^2 \right\},$$

where τ is the ATT you care about, α_i are the unit-fixed effects, and β_t are the time-fixed effects. In this formulation, the unit effects capture the difference in intercepts for each unit while the time effects capture the general trend across both treated and control units. The main assumption behind the DID method is that the treated and untreated have the same Y_0 trend:

$$\Delta Y(d)_i \perp D$$

Synthetic Controls Revisited

Next, let's see how you can recast the synthetic control estimator into something that resembles the preceding DID

formulation. Interestingly enough, you can write the SC estimator as solving the following optimization problems:

$$\hat{\tau}^{sc} = \underset{\beta, \tau}{\operatorname{argmin}} \left\{ \sum_{n=1}^N \sum_{t=1}^T (Y_{it} - \beta_t - \tau W_{it})^2 \hat{\omega}_i^{sc} \right\},$$

where the weights for the control units $\hat{\omega}_i^{sc}$ are obtained by optimizing the synthetic control objective you saw at the beginning of the chapter. Since the preceding formulation of the SC objective is defined for all units, not just the control, you also need to think about the treatment units' weight. Here, since you care about the ATT, they are simply N_{tr}/N (uniform weighting).

To verify that this new formulation is indeed equivalent to the one you learned earlier, let's compare the two. First, estimate SC as you've done so far and compute the ATT:

```
In [24]: sc_model = SyntheticControl()
         sc_model.fit(y_pre_co, y_pre_tr.mean(axis=0))

         (y_post_tr.mean(axis=1) - sc_model.predict(
```

```
Out[24]: 0.0033467270830624114
```

Next, let's add these synthetic control weights to the original marketing data frame, before the matrix reshaping. To do that, you can create a unit weights data frame that maps each control city to its weight:

```
In [25]: unit_w = pd.DataFrame(zip(y_pre_co.columns,
                                   unit_weights),
                                columns=["city", "unit_weight"])

unit_w.head()
```

	city	unit_weight
0	ananindeua	-1.649964e-19
1	aparecida_de_goiania	-7.047642e-21
2	aracaju	4.150540e-19
3	belem	-3.238918e-19
4	belford_roxo	-5.756475e-19

Then, you can merge this unit weight data frame into the original marketing data frame using city as the key. This will

leave the treatment unit with NaN weights. You can fill those up with the average of the treatment dummy, which is N_{tr}/N .

I'll also take this opportunity to create the W_{it} variable, by multiplying $D_i * Post_t$:

```
In [26]: df_with_w = (df_norm
                .assign(tr_post = lambda d: d["tr"] * d["post"])
                .merge(unit_w, on=["city", "year"])
                .fillna({"unit_weight": d["unit_weight"].mean()}))

df_with_w.head()
```

	app_download	population	city
0	3066.0	12396372	sao_pau
1	2701.0	12396372	sao_pau
2	1927.0	12396372	sao_pau
3	1451.0	12396372	sao_pau
4	1248.0	12396372	sao_pau

Finally, you can run weighted OLS with time-fixed effects, following the alternative synthetic control formulation from earlier. Just be careful to remove the rows with very small weights, otherwise you might run into some errors while trying to run this regression:

```
In [27]: mod = smf.wls(
            "app_download_pct ~ tr_post + C(data)
            data=df_with_w.query("unit_weight>0")
            weights=df_with_w.query("unit_weight>0")
        )

        mod.fit().params["tr_post"]
```

```
Out[27]: 0.00334672708306243
```

Indeed, the ATT obtained here is exactly the same as the one you got earlier, which shows that both synthetic control formulations are equivalent. But more importantly, it's easier to compare the new SC formulation with the TWFE formulation of DID. First, it looks like SC has time-fixed effects, but not unit-fixed effects. Meanwhile DID has both time- and unit-fixed effects, but no unit weights. This suggests a merger between the two models that include elements from both SC and DID:

$$\hat{\tau}^{sdid} = \underset{\mu, \alpha, \beta, \tau}{argmin} \left\{ \sum_{n=1}^N \sum_{t=1}^T (Y_{it} - (\mu + \alpha_i + \beta_t + \tau D_{it}))^2 \hat{\omega}_i \right\}$$

And while you are at it, why only weigh the units? Always remember the end goal here: to estimate $E[Y_0|Post = 1, Tr = 1]$. The purpose of unit weights is to use the control units to approximate the treated units. But there is also a time dimension here, which means you could also use weights in the pre-treatment periods to better approximate the post-treatment period. This would give you the following SDID formulation:

$$\hat{\tau}^{sdid} = \underset{\mu, \alpha, \beta, \tau}{argmin} \left\{ \sum_{n=1}^N \sum_{t=1}^T (Y_{it} - (\mu + \alpha_i + \beta_t + \tau D_{it}))^2 \hat{\omega}_i \hat{\lambda}_t \right\},$$

where $\hat{\lambda}_t$ are time weights.

Estimating Time Weights

Remember how, in order to get the unit weights, you've regressed the average outcome of the treated units on the outcome of the control units, both in the pre-treatment period?

$$\begin{aligned} \hat{\omega}_i^{sc} &= \underset{\omega}{argmin} \quad ||\bar{\mathbf{y}}_{pre,tr} - \mathbf{Y}_{pre,co} \omega_{co}||^2 \\ \text{s.t.} \quad &\sum \omega_i = 1 \text{ and } \omega_i > 0 \quad \forall i \end{aligned}$$

Well, to get time weights, you just need to transpose the $\mathbf{Y}_{pre,co}$ and regress it on the average outcome of the control

on the *post-treatment* period:

$$\hat{\lambda}_t^{sc} = \underset{w}{\operatorname{argmin}} \quad \|\bar{\mathbf{y}}_{pre,co}' - \mathbf{Y}_{pre,co}' \lambda_{pre}\|^2$$
$$\text{s.t.} \quad \sum \lambda_i = 1 \text{ and } \lambda_i > 0 \quad \forall i$$

But there is an additional catch here. Remember how SC doesn't allow for extrapolations? This would be a problem if you had some kind of trend in the outcome. If that was the case, the average post-treatment period would have a higher or lower outcome than all the pre-treatment periods and you would need to extrapolate in order to get a good fit. For this reason, the SDID formulation allows for an intercept shift λ_0 when finding the time weights:

$$\hat{\lambda}_t^{sc} = \underset{w}{\operatorname{argmin}} \quad \|\bar{\mathbf{y}}_{pre,co}' - (\mathbf{Y}_{pre,co}' \lambda_{pre} + \lambda_0)\|^2$$
$$\text{s.t.} \quad \sum \lambda_i = 1 \text{ and } \lambda_i > 0 \quad \forall i$$

Fortunately, it is fairly easy to modify the

`SyntheticControl` code to optionally fit the intercept, using a `fit_intercept` parameter. First, you'll create an intercept column that is always 1 if

`fit_intercept=True` and zero otherwise. You can take advantage of the fact that multiplying `True*1=1` in Python. Then, you'll prepend this column into

`y_pre_co` and use that in the objective function. Also, when building the constraints, you don't want to include the intercept. In the end, you'll get rid of the intercept parameter, returning only the units' weights.

I won't show the entire code because it is fairly repetitive, but here is just the part that changes:

```
# add intercept
intercept = np.ones((y_pre_co.shape[0],
X = np.concatenate([intercept, y_pre_co
w = cp.Variable(X.shape[1])

objective = cp.Minimize(cp.sum_squares(2
constraints = [cp.sum(w[1:]) == 1, w[1:]

problem = cp.Problem(objective, constra

self.loss_ = problem.solve(verbose=False
self.w_ = w.value[1:] # remove intercept
```

Once you deal with that, you can move on to estimate the time weights:

```
In [28]: time_sc = SyntheticControl(fit_intercep

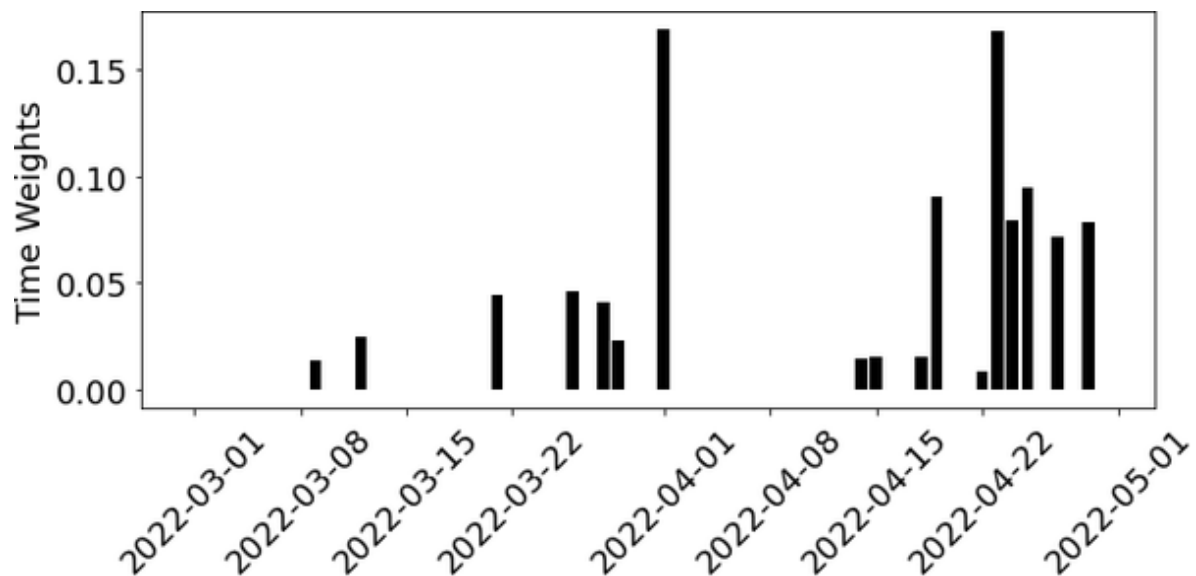
time_sc.fit(
    y_pre_co.T,
    y_post_co.mean(axis=0)
)

time_w = pd.DataFrame(zip(y_pre_co.index
                        columns=["date
```

```
time_w.tail()
```

	date	time_weight
56	2022-04-26	-0.000011
57	2022-04-27	0.071965
58	2022-04-28	-0.000002
59	2022-04-29	0.078350
60	2022-04-30	0.000002

I also already stored the weights in a data frame, which you'll use later. You can also plot these weights to see how the pre-treatment time periods are being used to reconstruct the average outcome of the control in the post-treatment period:



Synthetic Control and DID

OK. So you have weights for the pre-intervention period and weights for all the units. All there is left to do is join these pieces into a final estimator. You can start from the previously defined data frame `df_with_w` and join in the time weights data frame, using `date` as the key. Since `time_w` has weights only for the pre-intervention period, you need to fill in the post-intervention time weights with T_{post}/T (also uniformly weighting them). Finally, multiply both λ_t and ω_i and you are good to go:

```
In [29]: scdid_df = (
    df_with_w
    .merge(time_w, on=["date"], how="left")
    .fillna({"time_weight": df_norm["post_weight"]})
    .assign(weight = lambda d: (d["time_weight"] * d["weight"])
```

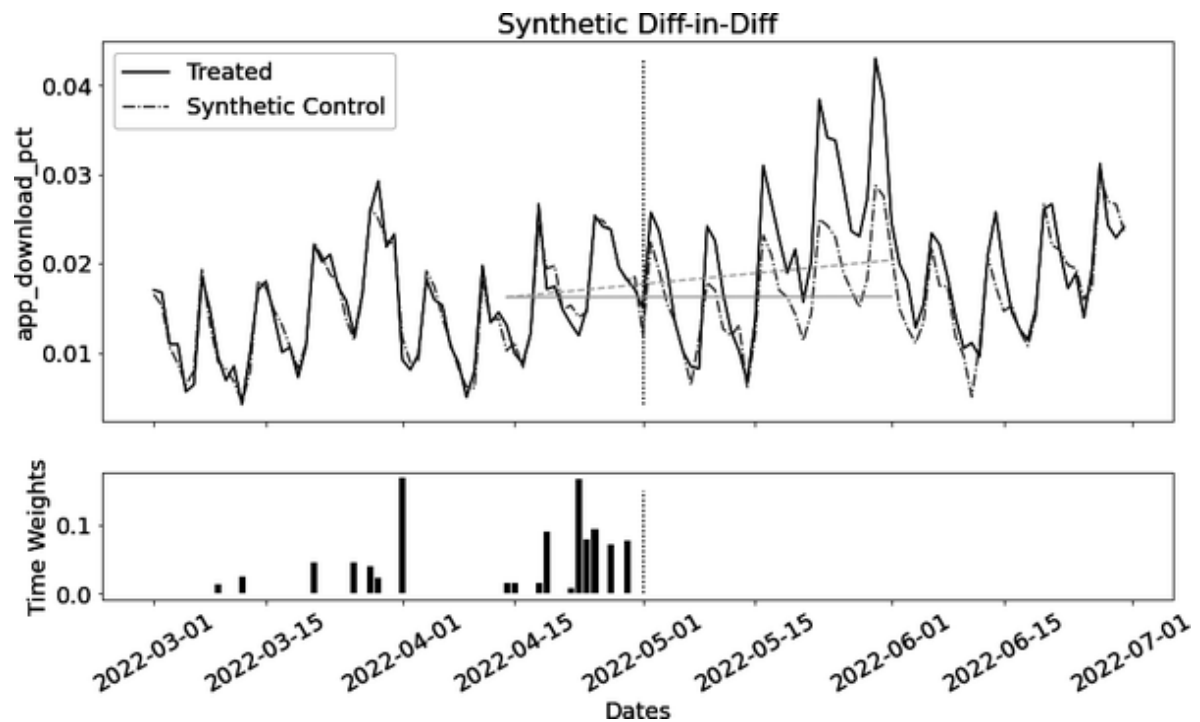
```
)
```

You can now run DID using the `scdid_df` data and weighted regression. The parameter estimate associated with $W_{it} = D_i * Post_t$ is the ATT estimate you care about:

```
In [30]: did_model = smf.wls(  
        "app_download_pct ~ treated*post",  
        data=scdid_df.query("weight>1e-10"),  
        weights=scdid_df.query("weight>1e-10").weight,  
        did_model.params["treated:post"]
```

```
Out[30]: 0.004098194485564245
```

To grasp what SDID is doing, you can plot the diff-in-diff lines for the treated units and the counterfactual trend (dashed line) obtained by using the synthetic control's trend and projecting it into the treated's baseline. The difference between the two is the ATT estimate you just estimated. I'm also plotting the time weights on a second plot. You can see how SDID uses mostly time periods that are closer to the post-intervention period:



This SDID estimate is a bit higher, but not much different from the ATT of the canonical SC. So, why is SDID interesting? The SC component of the estimator makes the DID's parallel trend assumption more plausible. It's much easier to get parallel trends if you first craft a synthetic control to mimic the treated units. As a result, SDID tends to have lower bias, compared to both DID and SC. Second, SDID also tends to have lower variance than both methods. The original paper has simulation results to show this, in case you are interested.

THE ORIGINAL SDID

This SDID estimator is a simplification on top of the original SDID estimator, which was proposed in the paper “Synthetic Difference in Differences,” by Dmitry Arkhangelsky et al. The paper proposes a slightly different optimization objective for the unit weights:

$$\hat{\omega}_i^{scdid} = \underset{\omega}{\operatorname{argmin}} \quad \|\bar{\mathbf{y}}_{pre,tr} - (\mathbf{Y}_{pre,co}\omega_{co} + \omega_0)\|_2^2 + \zeta^2 T_{pre} \|\omega_{co}\|_2^2$$
$$\text{s.t.} \quad \sum \omega_i = 1 \text{ and } \omega_i > 0 \quad \forall i$$

First, this objective also allows an intercept shift. The reason here is that the synthetic control doesn’t need to exactly match the treated unit, only its trend, since you’ll plug them in a DID model afterward. Second, they add an L2 penalty on the unit weights, which includes this new ζ term:

$$\zeta = (N_{tr} * T_{post})^{1/4} \sigma(Y_{it} - Y_{i(t-1)})$$

There is a complicated theoretical reason for this ζ that I won’t go into, but the main idea behind the additional L2 penalty is to make sure no single unit gets a disproportionately large weight. As a result, these weights tend to be more spread out across more units than that of the canonical synthetic control method.

The paper also proposed a new inference procedure, designed specifically for SDID. If that isn’t enough reasons to check it out, I don’t know what is.

Of course, there is no free lunch. By allowing intercept shifts, SDID removes the convexity guardrail from SC. Depending on the situation, you can view this as either good, since SDID allows more flexibility, or bad, since it also allows dangerous extrapolations.

Key Ideas

If there is one thing I want you to take out from this chapter is that you can have a model-based approach to estimating $E[Y(0)_t | D = 1, Post = 1]$: just fit a model to predict pre-treatment outcome of the treated on a bunch of equally pre-treatment time series and use that model to make predictions on the post-treatment period. Usually, those time series are the outcome of control units, and this approach amounts to a horizontal regression of the treated outcome on the control outcome, all in the pre-intervention period:

$$\hat{\omega}^{sc} = \underset{\omega}{\operatorname{argmin}} \quad ||\bar{\mathbf{y}}_{pre,tr} - \mathbf{Y}_{pre,co}\omega_{co}||_2^2$$

The result is a set of weights which, when multiplied by the control units, yields a synthetic control: a combination of control units that approximate the behavior of the treated units, at least in the pre-intervention period. If that approximation is good and generalizes into the post-intervention period, you can use it to estimate the ATT:

$$\widehat{ATT} = \mathbf{Y}_{post,tr} - \mathbf{Y}_{post,co}\widehat{\omega}_{co}$$

That is the basic idea. You can build up from it. For example, in the canonical synthetic control setting, you would add the constraints $\sum \omega_i = 1$ and $\omega_i > 0 \ \forall \ i$ or use something like lasso regression. But the basic idea remains: use the pre-intervention period to regress the pre-treatment outcome on very predictive time series and extend those predictions to the post-intervention period.

SEE ALSO

Yiqing Xu has a bunch of papers on generalizing synthetic control, as well as software implementing those approaches. To name a few, the paper “Generalized Synthetic Control Method: Causal Inference with Interactive Fixed Effects Models” also mixes qualities from both DID and synthetic control, generalizing this last one to variable treatment periods (staggered adoption design). In “Bayesian Alternative to Synthetic Control for Comparative Case Studies,” the authors also propose a Bayesian model to estimate counterfactuals.

RACTICAL EXAMPLE

CAUSAL IMPACT

Google’s research team capitalized on the main idea behind synthetic controls to build the `causalimpact` library. They use Bayesian structural time-series models to estimate the counterfactual time series for $E[Y(0)|D = 1]$ by using other time series that are themselves not affected by the treatment. The fact that the method is Bayesian also allows them to give uncertainty metrics quite naturally.
